



BITS Pilani

Pilani Campus

Instruction Set Architecture and Pipelining



Bandwidth and latency



- Latency (execution time): time to finish a fixed task
- Throughput (bandwidth): number of tasks in fixed time
- Different: exploit parallelism for throughput, not latency (e.g., bread)
- Often contradictory (latency vs. throughput)

Measuring and Reporting Performance



Execution time:

CPU time = Instruction count * Clock cycles per instruction
* Clock cycle time

“X is n times as fast as Y” will mean

Execution time_Y / Execution time_X = n

Since execution time is the reciprocal of performance, the following relationship holds:

$$n = \frac{\text{Execution time}_Y}{\text{Execution time}_X} = \frac{\frac{1}{\text{Performance}_Y}}{\frac{1}{\text{Performance}_X}} = \frac{\text{Performance}_X}{\text{Performance}_Y}$$

Speed up



Amdahl's Law:

$$\begin{aligned} \text{Speedup}_{\text{overall}} &= \frac{\text{Execution time}_{\text{old}}}{\text{Execution time}_{\text{new}}} \\ &= \frac{1}{(1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}}} \end{aligned}$$



Deeply pipelined machines



Many instructions/cycle



Out-of-order execution of instructions



Aggressive branch prediction techniques

Example3

Computers A and B implement the same ISA. Computer A has a clock cycle time of 250 ps and an effective CPI of 2.0 for some program and computer B has a clock cycle time of 500 ps and an effective CPI of 1.2 for the same program. Which computer is faster and by how much?

Solution

Assume each computer executes I instructions,

so CPU time $A = I \times 2.0 \times 250 = 500 \times I$ ps

CPU time $B = I \times 1.2 \times 500 = 600 \times I$ ps

A is faster by the ratio of execution times

$$\frac{\text{performance}_A}{\text{performance}_B} = \frac{\text{execution_time}_B}{\text{execution_time}_A} = \frac{600 \times I}{500 \times I} = 1.2$$

Layered view of computer systems.

Application
Compiler/libraries of macros and procedures
Operating system
Instruction set (ISA)
Computer architecture (organization)
Circuits (implementation of hardware functions)
Semiconductor physics

ISA



Development is very slow

- ISAs varied
- No. of operands
- Implied operands
- Operands may be stored in stack

Instruction types and opcodes



Four classes of instructions:

- **integer arithmetic/logic instructions:** opcodes for two's complement operations are ADD, SUB, MULT, and their unsigned equivalents are ADDU, SUBU, and MULTU
- **floating-point arithmetic instructions:** The purpose is to widen the range of representable numbers and to avoid arithmetic overflows and underflows. FADD, FMUL, FDIV.
- **memory transfer instructions:** LB, LH, LW, and LD; SB, SH, SW, and SD
- **control instructions.**

Instruction mixes



- a distribution of instructions is called an *instruction mix*.
- Two kinds of instruction mixes: static or dynamic.
- A static instruction mix is the distribution of instruction types in the program code.
- A dynamic instruction mix is the distribution of instruction types that are executed.
- In a dynamic mix one instruction at a given PC may be executed and counted multiple times (if it is part of a loop or a subroutine), whereas it is counted only once in the static mix.
- Static mixes are used to size instruction caches and memories.
- Dynamic mixes are much more helpful than static mixes as a primary guideline in designing the CPU; CPU design should focus on common case instructions in the dynamic instruction mix, i.e., instructions that consume the most time in the execution of a typical program.

Instruction operands



- Operands inside the CPU:
 1. Accumulator: an implicit operand inside the CPU
 2. Evaluation Stack: Operands are in a hardware structure accessed as a stack.
 3. Registers: managed by the compiler.
 4. Immediate operands: constants stored in the instruction.
- Memory operands:
 - Memory operands are pointed to by their *effective address*.
 - Addressing modes
 - Number of memory operands

Addressing modes



innovate

achieve

lead

Table 3.2 Examples of addressing modes in a register-based machine; (Ri) denotes “the value in Ri ”; $MEM[x]$ refers to the content of memory location at address x

Mode	Example	Meaning
Register-direct	ADD $R1, R2, R3$	$R1 \leftarrow (R2) + (R3)$
Immediate	ADDI $R1, R2, \#15$	$R1 \leftarrow (R2) + 15$
Displacement	LOAD $R1, \#20(R2)$	$R1 \leftarrow MEM[(R2) + 20]$
Register-indirect	LOAD $R1, (R2)$	$R1 \leftarrow MEM[(R2)]$
Memory-absolute	LOAD $R1, \#2000$	$R1 \leftarrow MEM[2000]$
Memory-indirect	LOAD $R1, @(R2)$	$R1 \leftarrow MEM[MEM[(R2)]]$
Post-increment	LOAD $R1, (R2) +$	$R1 \leftarrow MEM[(R2)]$ then $R2 \leftarrow (R2) + \text{size}$
Pre-decrement	LOAD $R1, -(R2)$	$R2 \leftarrow (R2) - \text{size}$ then $R1 \leftarrow MEM[(R2)]$
PC-relative	BEZ $R1, \#166$	$PC \leftarrow (PC) + 4 + 166$

Exceptions, traps, and interrupts



- Exceptions (as the word indicates) are very rare events that are triggered by the hardware and force the processor to execute an exception handler.
- caused by instruction, external interrupts, or hardware malfunctions.
- Interrupts:
 - **I/O device interrupts**
 - **Operating system calls**
 - **Instruction tracing and breakpoints**
 - **Integer or floating-point exceptions**
 - **Page faults**
 - **Undefined instruction**
 - **Hardware failure/alarm**
 - **Power failures**

CISC vs RISC



Characteristics of RISC –

- Simpler instruction, hence simple instruction decoding.
- Instruction comes undersize of one word.
- Instruction takes a single clock cycle to get executed.
- More general-purpose registers.
- Simple Addressing Modes.
- Fewer Data types.
- A pipeline can be achieved.

Characteristics of CISC –

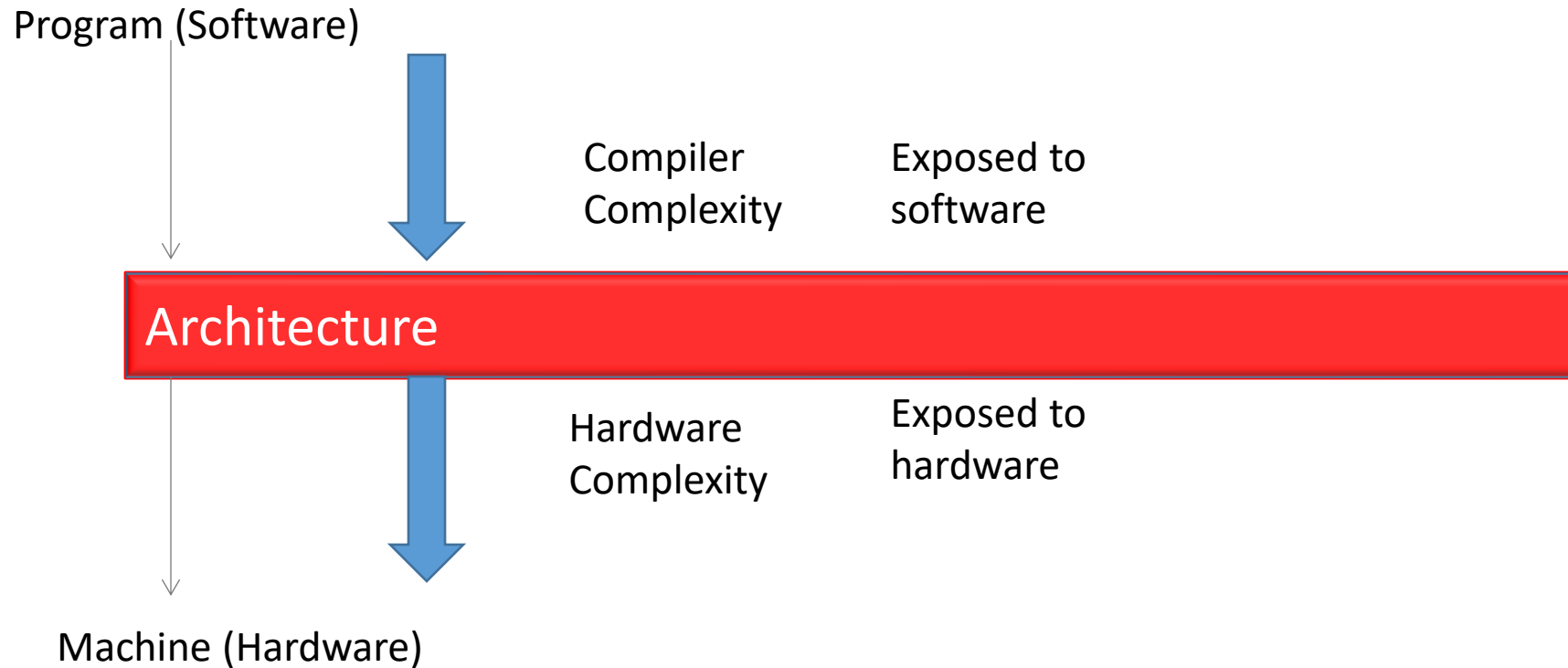
- Complex instruction, hence complex instruction decoding.
- Instructions are larger than one-word size.
- Instruction may take more than a single clock cycle to get executed.
- Less number of general-purpose registers as operations get performed in memory itself.
- Complex Addressing Modes.
- More Data types.

Dynamic – Static interface

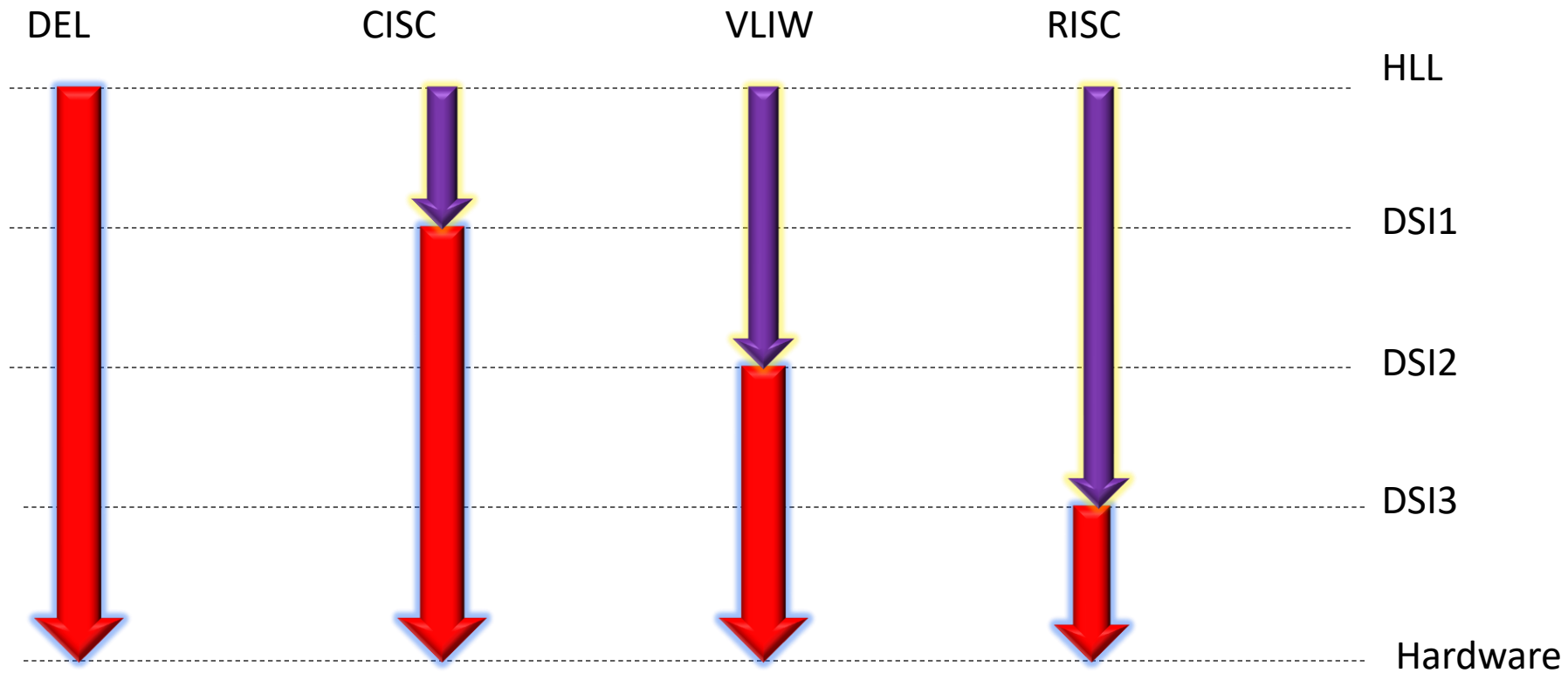


- Separates what is done
- Compile Time
 - Statically
- At run time
 - Dynamically

Dynamic – Static interface



Directed executable language



STATICALLY SCHEDULED PIPELINES



Table 3.5 Activity of each instruction in each stage of the pipeline

Instruction	I-fetch (IF)	I-decode (ID)	Execute (EX)	Memory (ME)	Write back (WB)
LW R1,#20(R2)	fetch; PC+=4	decode; fetch R2	compute address – (R2)+20	read	write in R1
SW R1,#20(R2)	fetch; PC+=4	decode; fetch R1 and R2	compute address – (R2)+20	write	–
ADD R1,R2,R3	fetch; PC+=4	decode; fetch R2 and R3	compute (R2)+(R3)	–	write in R1
ADDI R1,R2,imm.	fetch; PC+=4	decode; fetch R2	compute (R2)+imm	–	write in R1
BEQ R1,R2,offset	fetch; PC+=4	decode; fetch R1 and R2 compute target – address (PC)+offset	subtract (R1) and (R2); take branch if zero	–	–
J target	fetch; PC+=4	decode; take jump	–	–	–

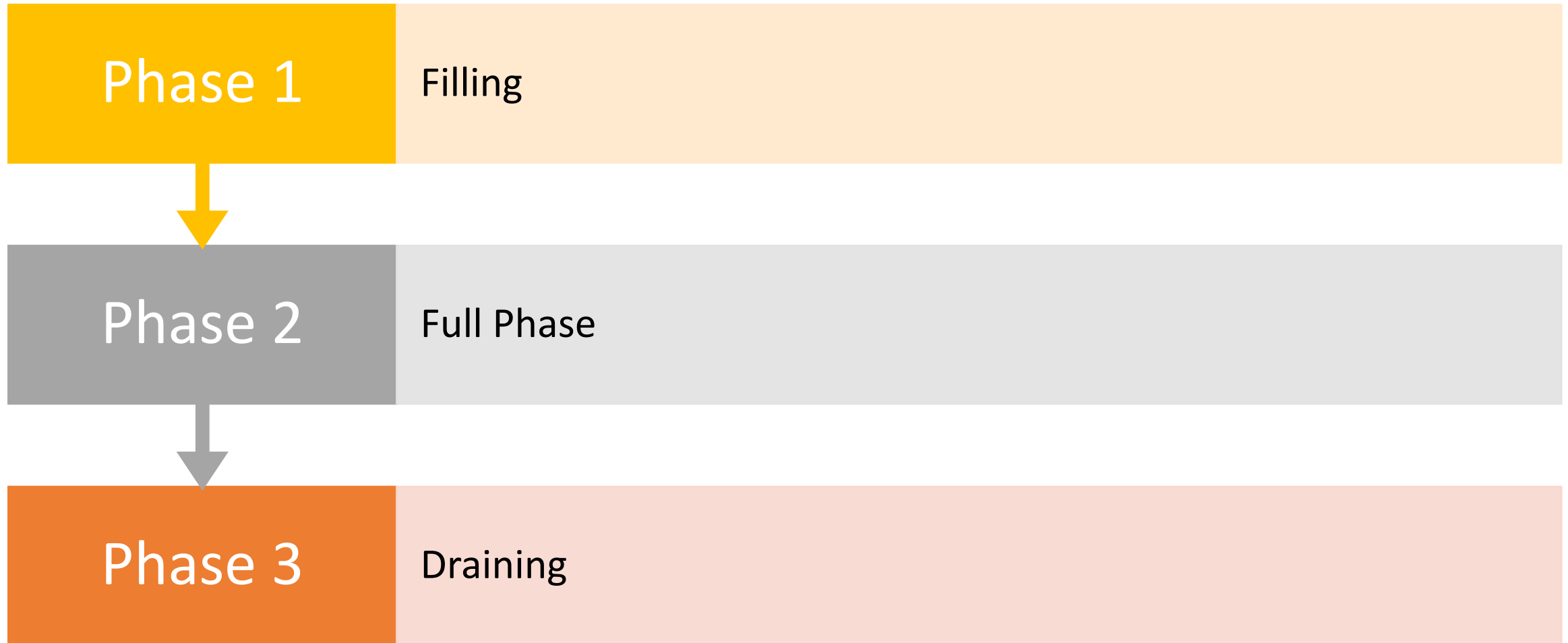
Classic 5 stage Pipeline

STATICALLY SCHEDULED PIPELINES



- basic pipeline: single, in-order issue
- first extension: multiple issue (superscalar)
- second extension: scheduling instructions for more Instruction Level Parallelism (ILP)
- option #1: dynamic scheduling (by the hardware)
- option #2: static scheduling (by the compiler)

Pipeline – N stages



5-stage pipeline



- The 5-stage pipeline has no mechanism to deal with cache misses. Thus, on a cache miss, the clock of the CPU is stopped and restarted after the cache miss is serviced.

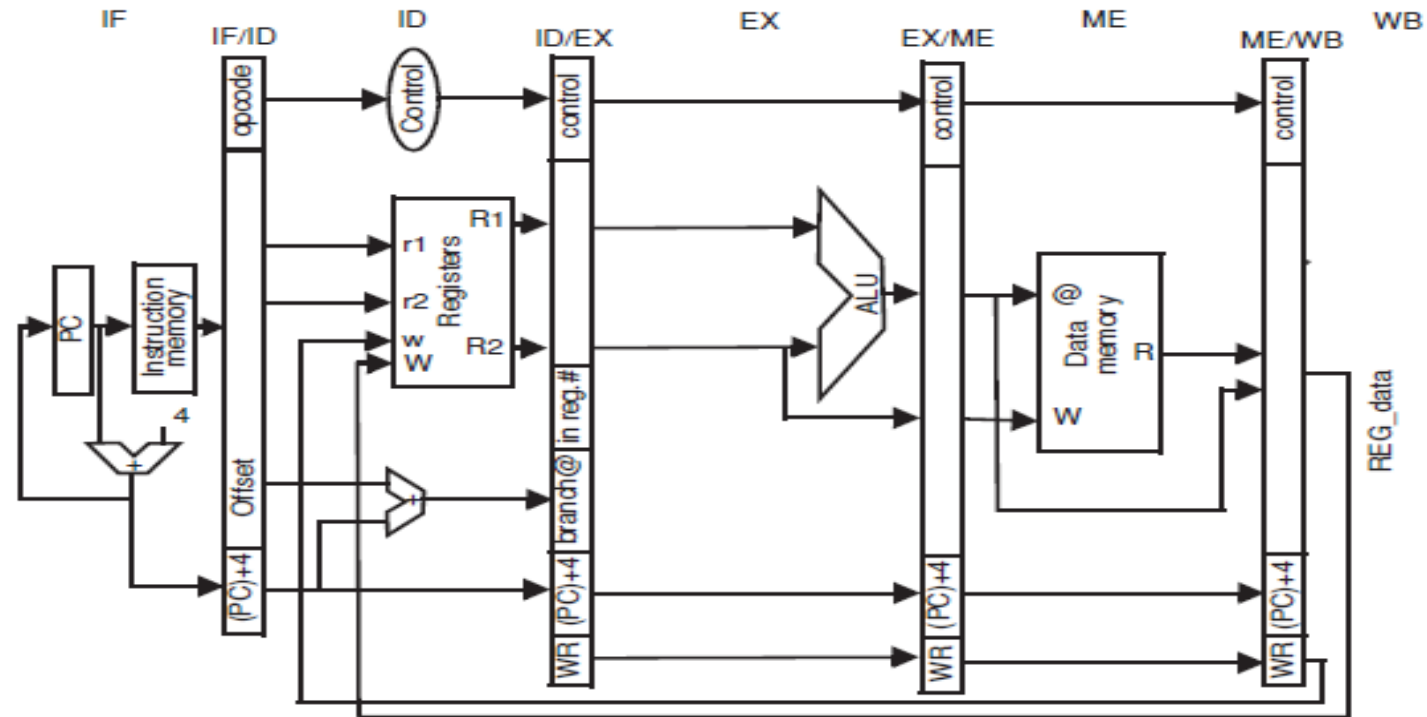
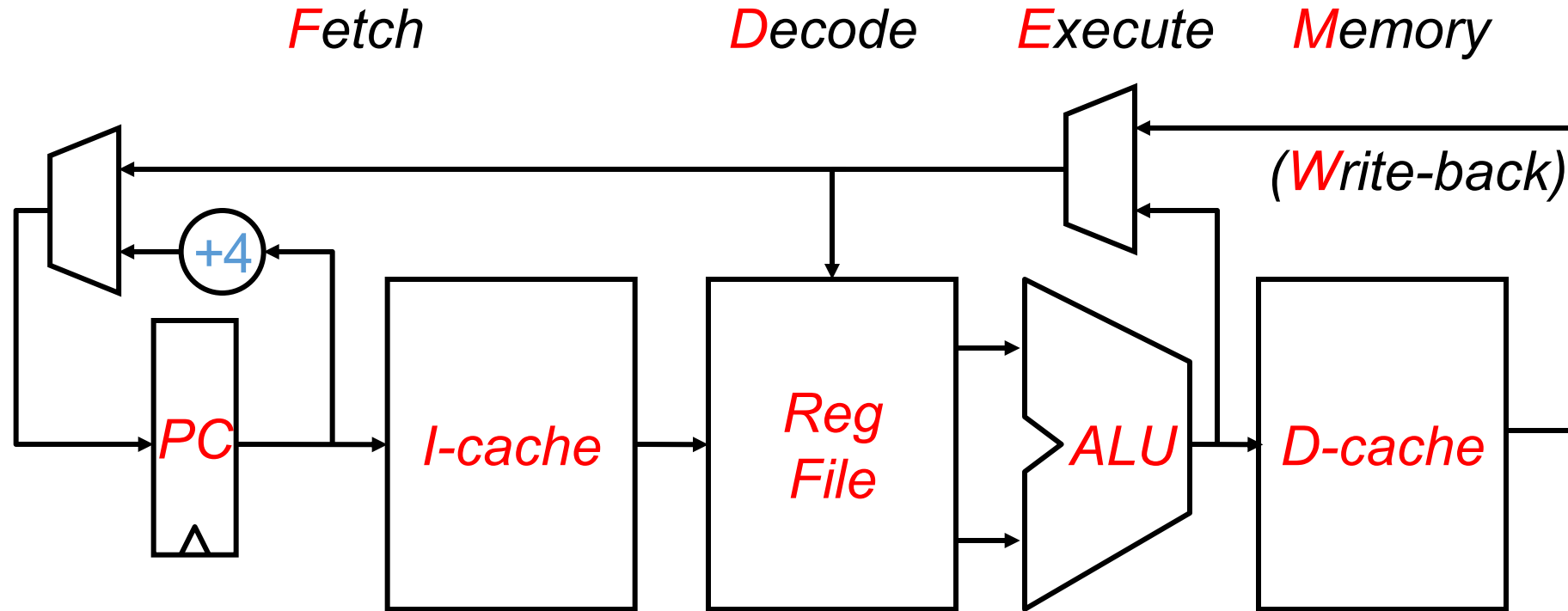


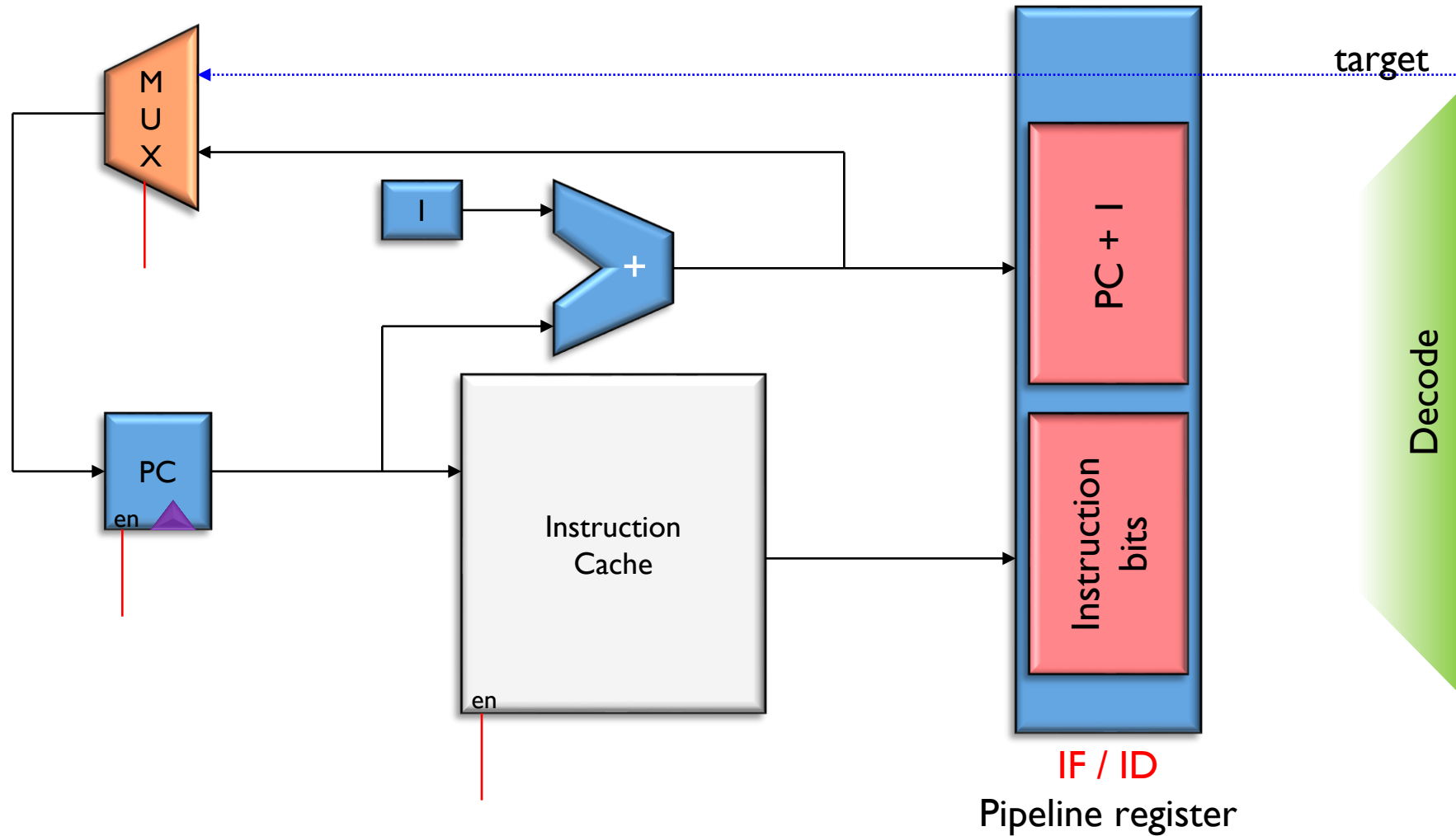
Figure 3.5. Basic 5-stage pipeline for independent loads, stores, and register-to-register instructions.

Pipeline Architecture

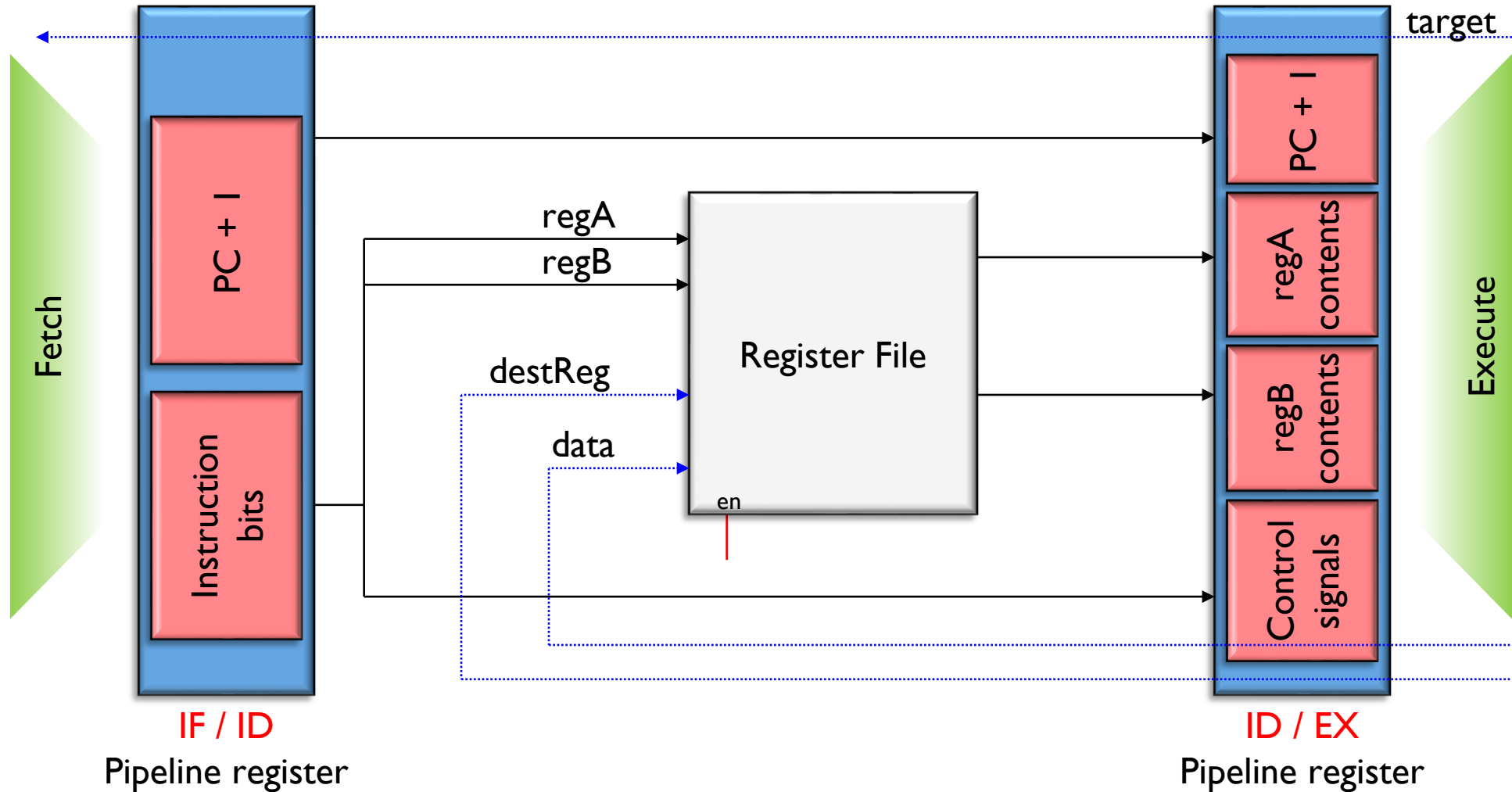
Processor Pipeline Review



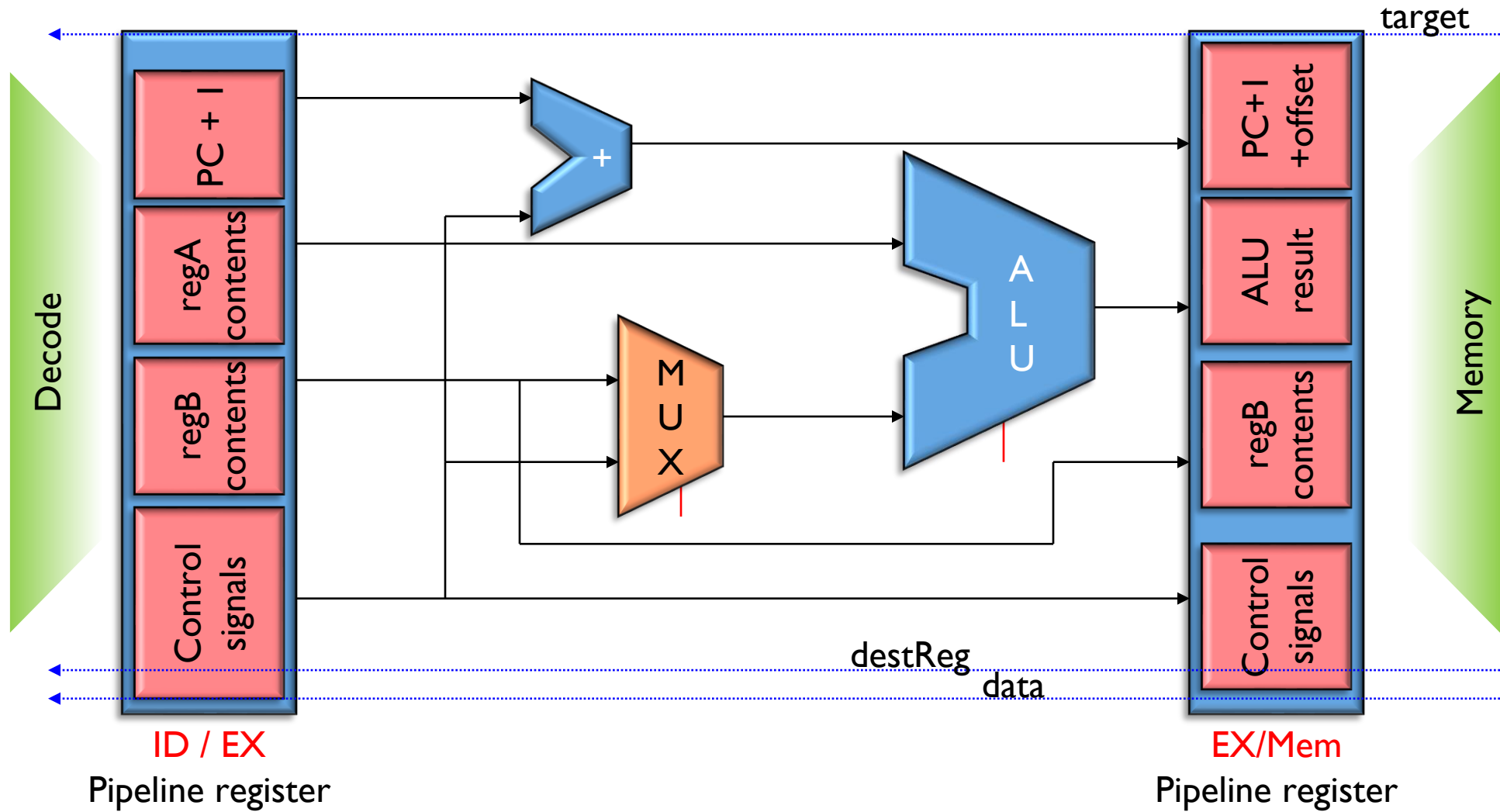
Stage 1: Fetch Diagram



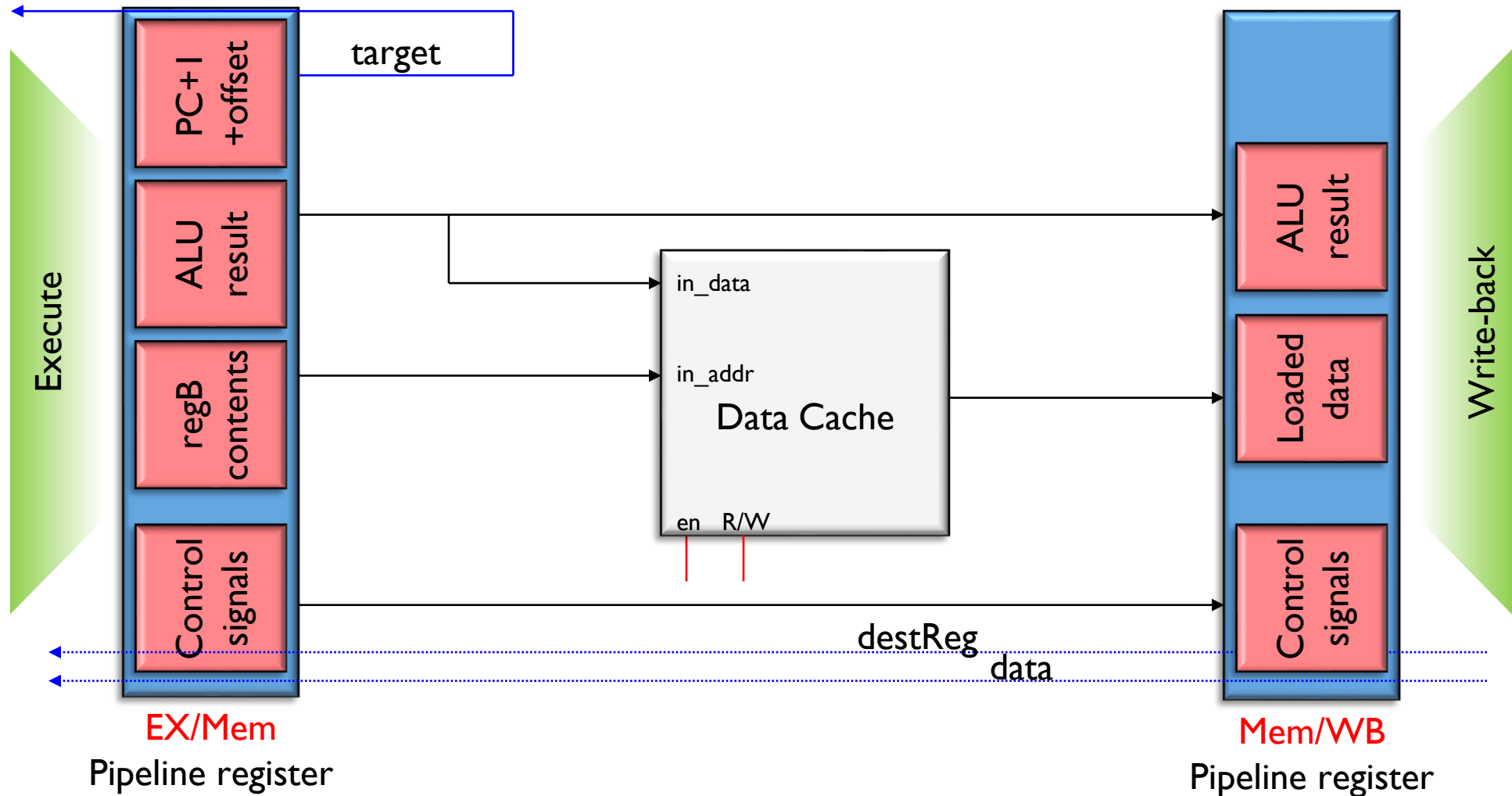
Stage 2: Decode Diagram



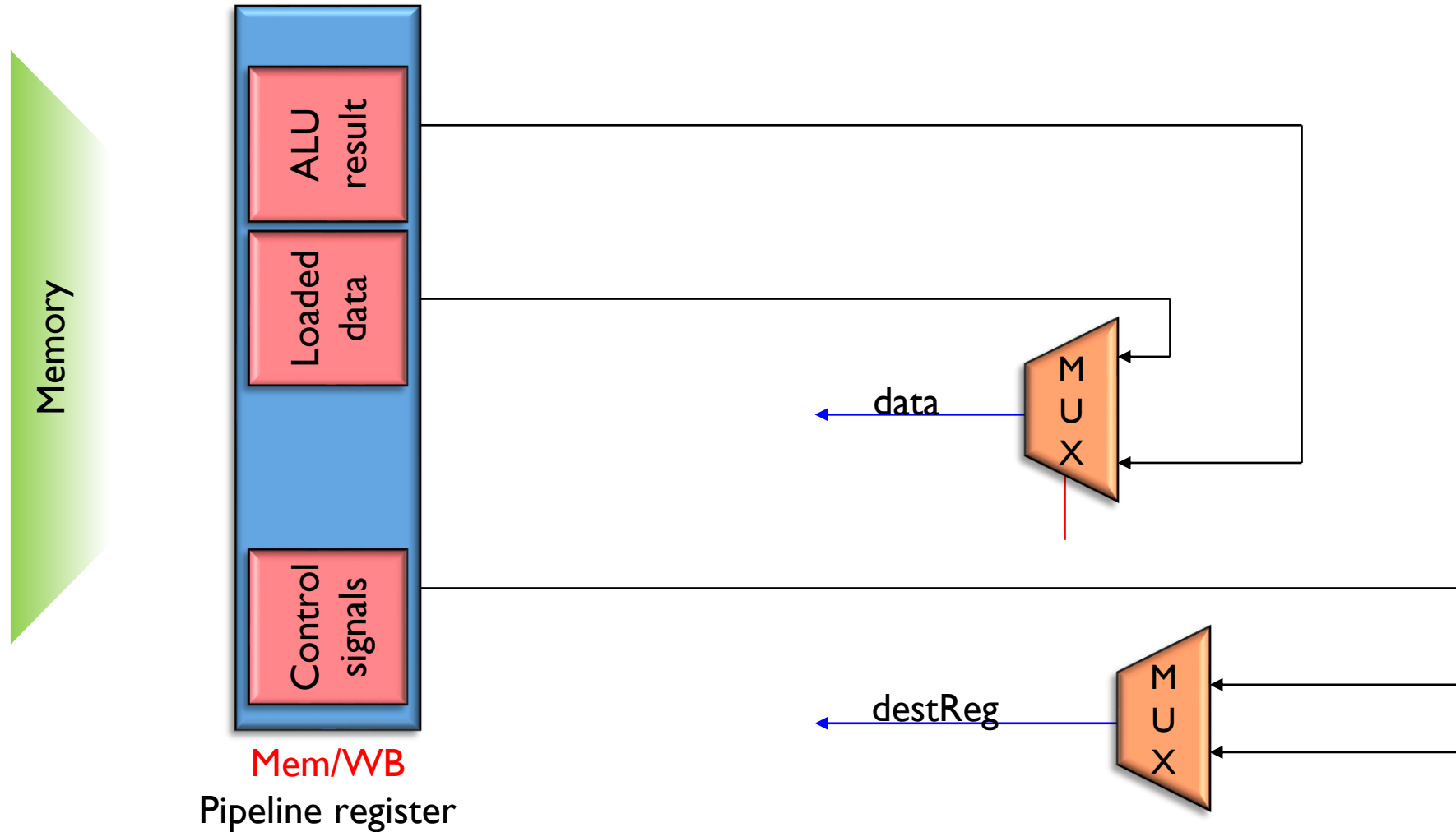
Stage 3: Execute Diagram



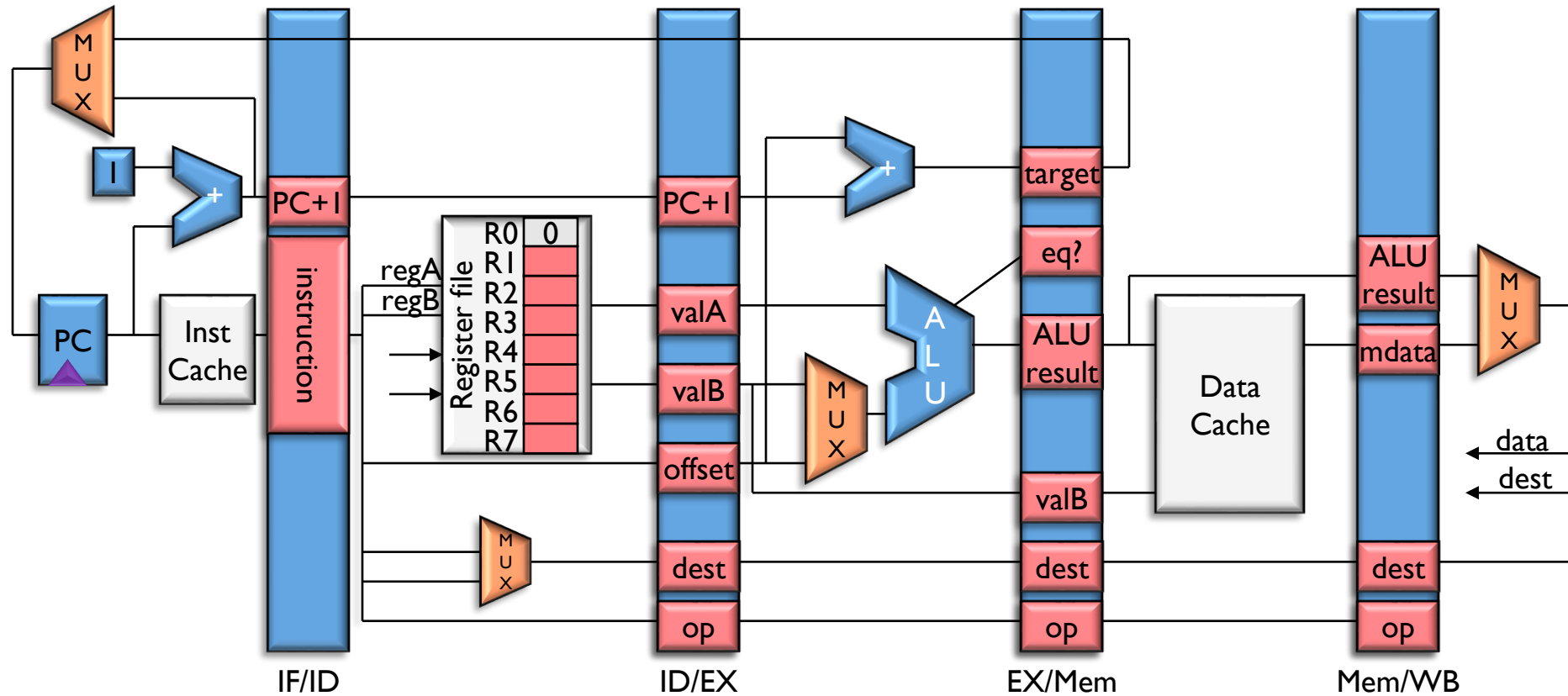
Stage 4: Memory Diagram



Stage 5: Write-back Diagram



Putting It All Together



Data Hazards



- When multiple instructions are in execution simultaneously (up to five in the 5-stage pipeline), data dependencies can cause data hazards, which must be resolved for correct operation.
- Note that dependencies are properties of the software only. Data hazards are caused by data dependencies and by the structure and instruction flow of the microarchitecture.

Data Hazards



RAW(read after write) A subsequent instruction needs the result of the current instruction. If the dependent instruction does not wait for the operand it may read a stale value (a RAW hazard).

Table 3.6 RAW hazards on registers after an ALU instruction

	Clock ==>	C1	C2	C3	C4	C5	C6	C7	C8	C9
I1	ADD R1,R2,R3	IF	ID	EX	ME	WB				
I2	ADDI R3,R1,#4		IF	ID	EX	ME	WB			
I3	LW R5,0(R1)			IF	ID	EX	ME	WB		
I4	ORI R6,R1,#20				IF	ID	EX	ME	WB	
I5	SUBI R1,R1,R7					IF	ID	EX	ME	WB

Data Hazards



- **WAR (write after read)** A subsequent instruction writes to an operand which is read by the current instruction. If the write is able to race ahead of the read, the current instruction reads a future value instead of the current value (a WAR hazard).
- **WAW (write after write)** A subsequent instruction writes into the same operand as the current instruction. If the second write is allowed to race ahead of the first write, then the final value of the operand is the first (oldest) value and therefore is stale (a WAW hazard).
- Data dependencies on memory operands do not cause hazards in the 5-stage pipeline, because all loads and stores execute in the ME stage and reach the ME stage in process order.
- There cannot be any WAW or WAR hazards on registers.
- WAW hazards are impossible because all instructions modifying the register file reach the WB stage in process order. The same argument also holds for WAR hazards because instructions always reach the decode stage, where registers are read before any subsequent instruction in process order.

Straight Line code

RAW/ true

- Load- use

```
l1: load  r1, a;  
l2: add   r2,r1,r1;
```



- Define –used

```
l1: mul   r1 ,r4, r5  
l2:add    r2,r1,r1;
```



Straight Line code

WAR /false/anti

I1: mul r1,r2,r3

I2: add r2,r4,r5

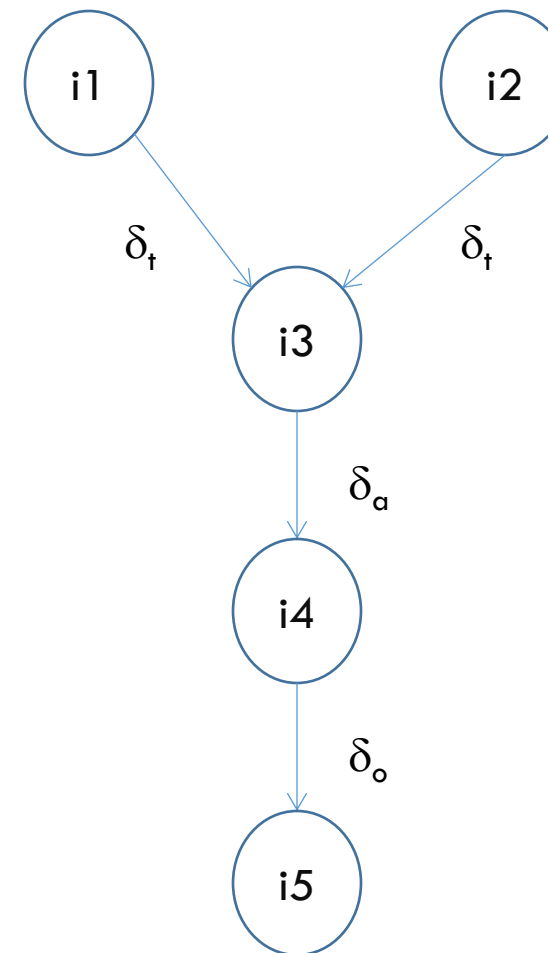
Straight Line Code

WAW/output

```
l1: mul    r1,r2,r3
l2: add    r1,r4,r5
```

Data dependency graph

load	r1,a
load	r2,b
add	r3,r2,r1
mul	r1,r2,r4
div	r1,r2,r4



Thank you